PBL PROBLEM MAP – DSL Data Quality Validator

a) Present the problem situation in 5–7 sentences

Modern companies load data into SQL databases from a wide variety of sources such as website forms, application logs, external APIs, CSV uploads, and internal services. Due to this heterogeneity, datasets almost inevitably contain quality issues, including incorrect formats, missing values, duplicates, and violations of business rules. These problems often remain hidden until the data is used for reporting, dashboards, or machine learning models. At that stage, inconsistencies lead to unstable metrics, unexpected results, and growing mistrust in data. Today, most teams address these issues using custom SQL queries or Python scripts that are difficult to maintain, reuse, and explain. Data quality rules are frequently undocumented, scattered across code, or exist only as implicit assumptions. Therefore, companies need a clear, simple, and repeatable way to define what "good data" means, identify problematic records, suggest safe corrections, and preserve original data.

b) Formulate the problem – 1 statement

There is no easy and repeatable way to define data quality rules for SQL tables, automatically detect violations, and obtain safe and explainable correction suggestions without writing custom code.

PBL PROBLEM MAP

1. Who has a problem?
   Data engineers, data analysts, data scientists, and developers responsible for maintaining reliable databases and analytics workflows.

2. Who is involved in the problem situation?
   All participants in the data lifecycle: applications and services that produce data, ETL engineers who move and transform it, analysts who interpret it, machine learning specialists who train models on it, and business users who rely on analytical results.

3. Who suffers from the problem?
   Technical specialists suffer first, as they spend many hours debugging pipelines and cleaning data manually. Then business users suffer because report preparation takes significantly more time, which delays decision-making and increases operational overhead.

4. Who should take part in solving the problem?
   At minimum, data engineers and domain or business experts should be involved. In larger setups, data product owners and compliance and/or security specialists may also participate.

5. What happened?
   Data stored in the database no longer follows expected quality rules, and important columns contain incorrect, missing, or inconsistent values.

6. What is the problem?
   Data quality checks are either missing or implemented through one-off scripts, which leads to late detection of issues and risky manual fixes.
7. What led you to identify the problem (reasons, causes, evidence, arguments, findings, survey results, etc.)?
   We observe frequent pipeline failures, reports that contradict each other, repeated manual data cleaning before every analysis, and increasing time spent investigating data-related issues. These observations are consistently confirmed by industry research on data engineering and analytics practices.
8. What are the causes that affect those involved?
   - Data originates from many heterogeneous sources
   - Business rules and data quality constraints are poorly formalized, scattered across code, or exist only as implicit knowledge
   - SQL-level constraints and checks are hard to document, audit, and communicate to non-technical stakeholders
   - Transformations and migrations occur without sufficient validation and visibility
   - Human input errors and inconsistent data formats
9. What are the needs of those targeted?
   - Ability to define validation rules without writing code
   - Clear and understandable explanations of detected issues
   - Reasonable and transparent suggestions for fixing invalid values
   - Safe application of fixes or creation of cleaned data copies
   - Repeatable checks with a complete audit trail
10. What are the resources (financial, human, logistical, time, etc.) necessary to solve the problem?
    Financial: minimal for a prototype, higher for production usage
    Human: 1–2 developers and one domain expert
    Logistical: access to test databases and representative datasets
    Time: moderate; the main challenge lies in designing robust rules rather than database connectivity
11. Where did the problem arise (location, area, stage of the process)?
    The problem appears throughout the data lifecycle, especially during ingestion and transformation stages. It becomes most critical immediately before analytics and machine learning tasks.
12. When did the problem arise?
    It appeared gradually, as soon as companies started systematically collecting and storing large volumes of data for operational, analytical, and decision-making purposes.
13. When was the problem identified?
    Typically when data issues begin to regularly break reports, dashboards, or pipelines, usually detected by data analysts before reaching business users.
14. When is it planned to be resolved?
    Within this project, the problem is addressed once a working MVP supports core validation rules and safe fix application. In practice, data quality improvement is an ongoing, iterative process.

15. Why is there a need to solve the problem?
Poor data quality leads to delayed decisions, reduced trust in analytics, slower workflows, higher operational costs, and potential regulatory risks.

16. How (in what way) can the problem be solved?
By introducing a declarative domain-specific language (DSL) that allows users to define data quality rules explicitly. The system automatically executes validations, presents understandable error reports, suggests sensible fixes, and applies changes only after user approval.

17. How will the elimination of the problem be determined?
Through measurable improvements such as fewer invalid values, fewer pipeline failures, reduced manual data cleaning, reproducible validation results, and complete audit logs.

18. How big is the problem?
The problem is large and widespread. Even a small percentage of invalid values can severely impact analytics when they affect identifiers, dates, monetary fields, or other critical columns.

19. How much is the quantity/quality/number of people affected?
    - Quantity: number of invalid cells and affected rows
    - Quality: reliability and trustworthiness of analytics and ML models
    - People: engineers and all decision-makers who rely on data

20. How long will it take to resolve the problem?
While the problem cannot be fully eliminated, its impact can be significantly reduced. By introducing a dedicated DSL, the most frequent and critical data quality issues can be detected and resolved within minutes or hours, depending on task complexity, which is considerably faster than manual scripting approaches. The development and initial deployment of the DSL is planned to be completed by June, according to the university project timeline. Further improvements and rule extensions can then be introduced incrementally.

21. How many parties need to be involved in resolving it?
A small team of 2–4 people is sufficient for a prototype. In production environments, coordination between data owners, engineers, and data consumers is required.

22. Why is a data quality validator needed at the database layer if validation should ideally happen before data insertion?
In real-world systems, it is impossible to fully control all data entry paths, and validation rules continuously evolve. As a result, even well-designed pipelines inevitably produce corrupted or inconsistent data, which must be detected and corrected directly within the database.

How do you see solving this problem (in terms of deliverable/product/result)?

The final deliverable is a DSL-based Data Quality Validator that:

1. Connects to SQL databases
2. Accepts clear, human-readable quality rules for tables and columns
3. Detects violations and explains them in a transparent manner
4. Suggests reasonable fixes based on observed data patterns
5. Applies changes only after explicit user approval or generates cleaned and audit tables

The system acts as a formal and transparent data quality contract rather than a black-box automatic cleaner.

Conclusions, findings, reflections

- Data quality issues are permanent and can only be controlled, not fully eliminated.
- A DSL is effective because it transforms scattered scripts into readable, reusable, and auditable rules.
- Preserving original data and requiring human approval for changes is the strongest design choice.
- The main risk lies in overly aggressive automatic corrections, mitigated through transparency and user control.
- Overall, the solution provides a practical foundation for restoring trust in data used for analytics and machine learning.